CS3210 Assignment 2

Leong Yuhao

February 2026

1 Implementation

Our algorithm is centered on representing each nucleotide as a 4-bit nibble. Compared to the classic 8-bit ASCII character representation, this approach provides twice as much information per unit of memory. Bit-level parallelism is also utilized, as 8 nucleotides is packed into a single `uint32_t` word for efficient bit-based matching.

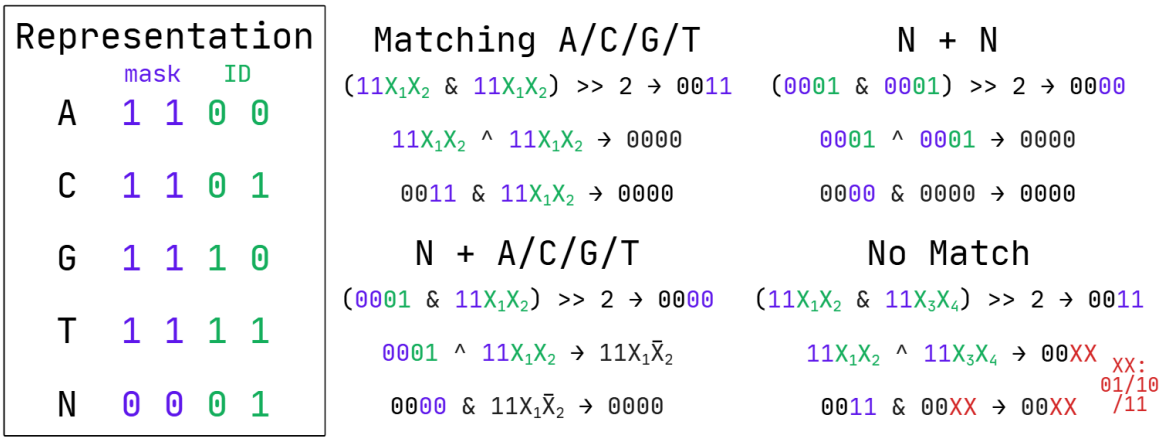

Figure 1: 4-bit matching: a final value of `0000` indicates a match. Note that an actual `uint32_t` word is 8 times as long, but the logic remains consistent.

Before matching, our algorithm first calls `kerPreprocessAllSigs`, which flattens and converts all signatures into a single `uint32_t` array. The algorithm then loops across all samples. At each iteration, we run `kerPreprocessSample`, which converts the sample sequence into a `uint32_t` array for bitwise matching with the converted sample. Then `kerPotentialMatchas` is called to identify potential matches between the current sample and all signatures. It should be noted that this is the main bottleneck of the program, accounting for more than 99% of the GPU's execution time. Each thread compares a 3-word window of the sample with the first 3 words of all signatures, corresponding to 24 nucleotides, and records each potential match into a vector. At the end of this kernel's execution, the program can proceed to the next iteration early if no potential matches are found. Otherwise, `kerConfirmMatchas` is called to verify each match and return the corresponding Phred scores. Simultaneously, `kerCalcIntegrity` calculates the integrity of the sample. Afterwards, the CPU calculates the greatest Phred score of each signature and places the `MatchResult` into the `matches` vector.

1.1 Detail of Kernels

1.1.1 `kerPreprocessAllSigs`

We set `blockDim.x = 1024` and `gridDim.x = num_sigs`, where each block is assigned a signature, and the block dimension is maximized according to the GPU specifications. All threads first perform a coalesced load of the entire signature into shared memory. At each iteration of the main loop, each thread loads 8 consecutive characters from shared memory and compresses them into a single `uint32_t` word, before incrementing its word position by 1024, ensuring all words are covered.

1.1.2 `kerPreprocessSample`

We set `blockDim.x = 1024` and `gridDim.x` = $\lceil$`total_words / blockDim.x`$\rceil$, where `total_words` is the length of the converted `uint32_t` sequence. This ensures work is evenly spread amongst all blocks. Each thread simply converts 8 characters into a single `uint32_t` word.

1.1.3 `kerPotentialMatchas`

This is the **critical** part of our algorithm, as the overwhelming majority of execution time is spent here. We set `blockDim.x = 1024` and `gridDim.x = 2 * num_SPs`. The value 2 is obtained from the max threads per SM (2048) divided by the block dimension, aimed to maximize occupancy. Shared memory is used to load all truncated signature info (first 3 words + length of each signature) and words from the sample. At each iteration of the main loop, all threads load a 131-word chunk of the sample into shared memory, derived from 1024 character offsets / 8 words per character + 3 words window. At each iteration, each thread compares a window of 24 characters from the sample with each signature. The windows overlap, as thread 0 has the window 0-23, thread 1 has the window 1-24, and so on. If a potential match is found, a thread records the character position of the sample along with the signature ID, by retrieving a write index atomically. At the end of the loop, each thread increments its character index by 1024.

1.1.4 `kerConfirmMatchas`

We set `blockDim.x = 1024` and `gridDim.x = num_pms`, where each block is assigned a potential match to verify. At each iteration of the first loop, each thread matches a word of the signature with the sample. If a no-match is found along the way, the block exits immediately. Otherwise, if the entire sequence matches, the block performs a parallel tree sum within shared memory to calculate the Phred score of the matching sample portion.

1.1.5 `kerCalcIntegrity`

We set `blockDim.x = 1024` and `gridDim.x = 2 * num_SPs - num_pms`. Each block performs a parallel tree sum within shared memory to calculate the Phred score of a portion of the sample, which is atomically added to the final sum. As this kernel runs simultenrously with `kerConfirmMatchas`, we also deploy a separate stream.

2 Performance Evaluation

Unless otherwise specified, the following parameters were set by default: `num_sigs = 210, avg_sig_len = 6500, num_samples = 400, avg_sample_len = 1500000, max_viruses = 10, percent_virus = 0.01, n_ratio = 0.1`. Min and max sample lengths were scaled to 0.667 and 1.333 of the average. Min and max signature lengths were scaled to 0.462 and 1.538 of the average. 'N' ratio was kept the same for every signature and sample pair. We fix min viruses, min Phred and max Phred at 1, 0 and 93 respectively. We obtained our timing measurements directly from the program output. Average and standard deviation were taken from 10 consecutive runs. Data for all experiments is provided in Appendix B.

2.1 Varying number of samples

For this experiment, we assign `num_samples` the values `[200 400 800 1600 3200]`, and fix `num_sigs = 840`. The execution time of our program scales linearly with the number of samples, where doubling the input size leads to approximately double the execution time. This is expected since the algorithm iterates across all samples in the main loop. We also deduce the additional overhead to be approximately 0.3-0.4s for all inputs and both GPU types.

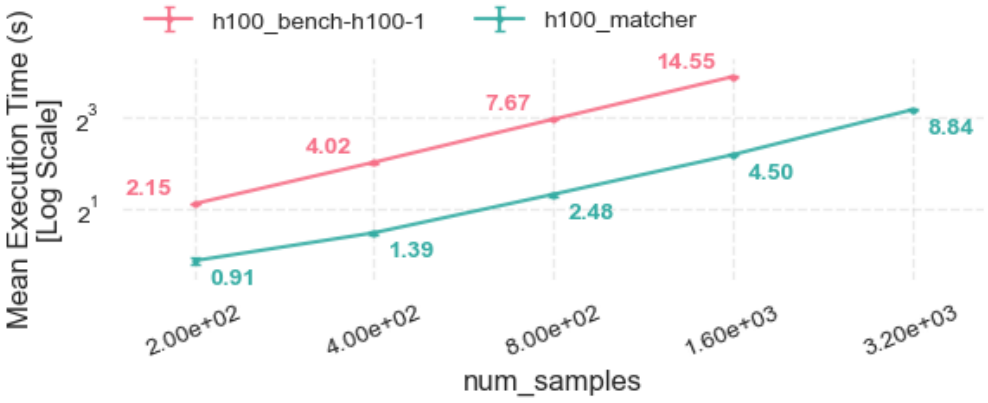

Figure 2: Varying number of samples, H100 (mean ± SD)

2.2 Varying average sample length

For this experiment, we assign `avg_sample_len` the values `[375000 750000 1500000 3000000 6000000]`, and fix `num_sigs = 840`. Similar the previous experiment, the execution time of our program scales linearly with the average sample length, which is expected since the main kernel `kerPotentialMatchas` processes each entire sample. Likewise, the overhead is also roughly 0.3-0.4s.

2.3 Varying number of signatures

For this experiment, we assign `num_sigs` the values `[105 210 420 840 1680]`. The execution time of our program scales linearly with the input size, which might not be apparent on the graph due to a relatively large overhead (roughly 0.5s) for a small input sizes along with logarithmic axes. This is consistent with `kerPotentialMatchas` matching each window in the sample against all signatures.

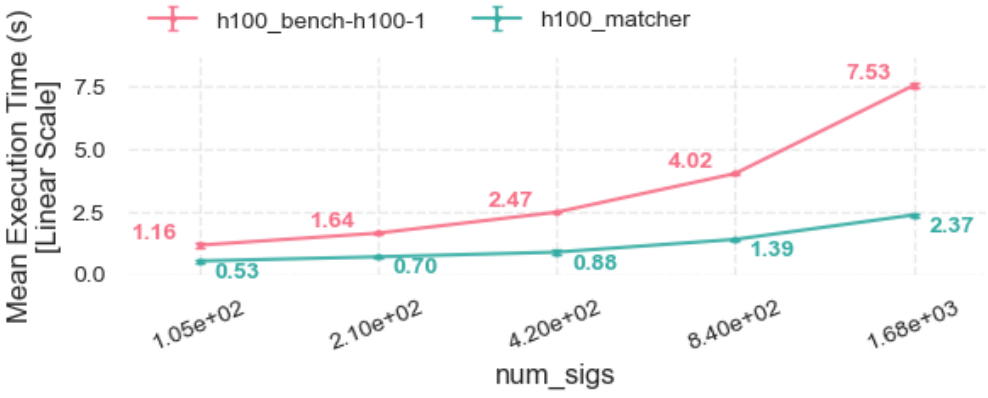

Figure 3: Varying number of signatures, H100 (mean ± SD)

2.4 Varying average signature length

For this experiment, we assign `avg_sig_len` the values `[1625 3250 6500 13000 26000]`. The execution time of our program does not appear to have a correlation with the average signature length. However, it is known that our algorithm does indeed scale with the signature length due to the logic in `kerPreprocessAllSigs` and `kerConfirmMatchas`. It is expected that with a greater `percent_virus` or `max_viruses`, the correlation would be more apparent.

2.5 Varying maximum number of viruses

For this experiment, we assign `max_viruses` the values `[3 10 30 100 300]`. The execution time of our program does not appear to have a correlation with the maximum number of viruses. However, it is known that our algorithm does indeed scale with the number of viruses present in each sample as `kerConfirmMatchas` assigns a block to each virus signature. However, the time taken for a single `kerPotentialMatchas` call is 100-1000 times longer than `kerConfirmMatchas`, and the maximum number of viruses is bounded by the ratio of the sample length to the signature length. To meaningfully observe this correlation, variables like `avg_sig_len` or `percent_virus` must also be raised.

2.6 Varying percentage with virus

For this experiment, we assign `percent_virus` the values `[0.01 0.03 0.1 0.3 1]`. Similar to the above, the execution time of our program does not appear to have a correlation with the percentage of samples with virus. Since increasing this percentage would increase the amount of virus signatures present across all signatures, we can once again deduce that other parameters must also be raised to measure the effect of this correlation.

2.7 Varying 'N' ratio

For this experiment, we assign `n_ratio` the values `[0.05 0.1 0.2 0.4 0.8]`. The execution time of our program does not appear to have a correlation with the 'N' ratio. However, from our algorithm, it can be said that the relationship is exponential when the ratio is close to 1. When the difference between the current 'N' ratio and 1 is halved, the number of 'A/C/G/T' is effectively halved, leading to double the false positives returned by `kerPotentialMatchas` and subsequently needing to be verified by `kerConfirmMatchas`.

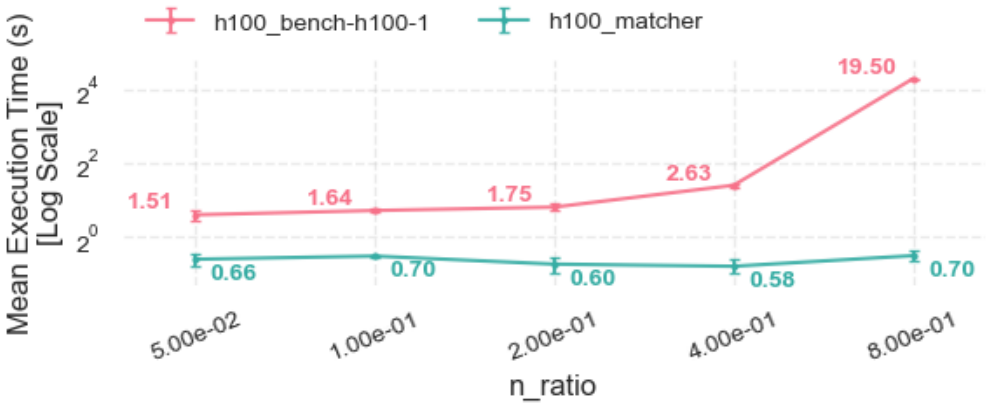

Figure 4: Varying 'N' ratios, H100 (mean ± SD)

3 Optimizations

3.1 Optimization 1 (`matchar` → `matcharv2`)

Our original implementation for `kerPotentialMatchas` had each thread match a 96-bit window of the sample across all truncated signatures without first storing the truncated signatures into shared memory. This meant that global loads were in effect every time a warp moved to the next signature. Furthermore, Nsight Compute reported a low achieved occupancy of 46.29% for said kernel, with `gridDim.x = 256` and `blockDim.x = 256`. To counteract these issues, we assigned 2 dimensions to the block and grid size, aimed to increase the total number of threads. We let `gridDim.x = 256, gridDim.y = 4, blockDim.x = 16, blockDim.y = 16`. The signatures were partitioned into `gridDim.y * blockDim.y = 64` sections of up to 16 signatures each. Within each section, the first 96 bits of each signature were then loaded into shared memory. We achieve a speedup of up to 1.4 for bigger inputs.

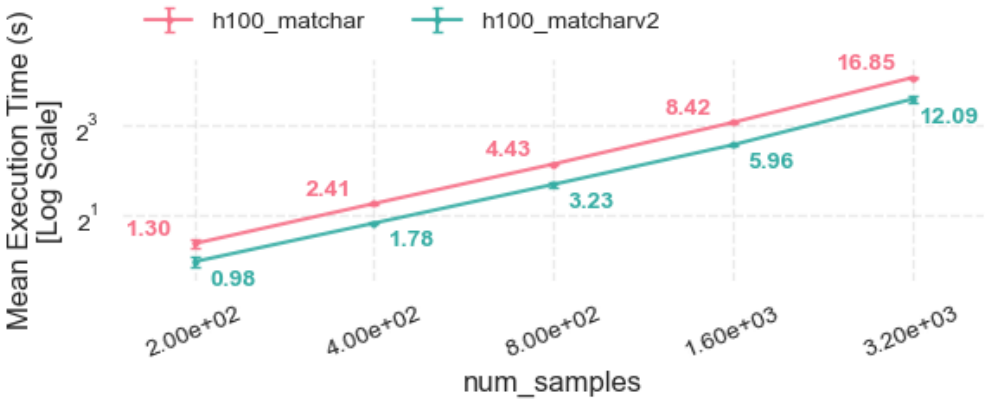

Figure 5: Varying number of samples, H100, `matcharv2` vs `matchar` (mean ± SD)

3.2 Optimization 2 (`matcharv2` → `matcher`)

Our first optimization still suffered from the issues 'Small Grid' (not making full use of SM) and 'Tail Effect' (unoptimal grid and block size) in Nsight Compute. To mitigate these issues, we decided to maximize the amount of blocks assigned to a streaming multiprocessor. For `kerCalcIntegrity` and `kerPotentialMatchas`, we scaled the grid size directly to the number of available SMs. Furthermore, we attempted to call the CUDA Occupancy API which returned a max block size of 1024 for the aforementioned kernels on the hardware, which was not included in the final code due to compiler warnings. We replaced it by hardcoding 1024 directly, as the hardware was already known. We achieve a speedup of approximately 1.35 for bigger inputs.

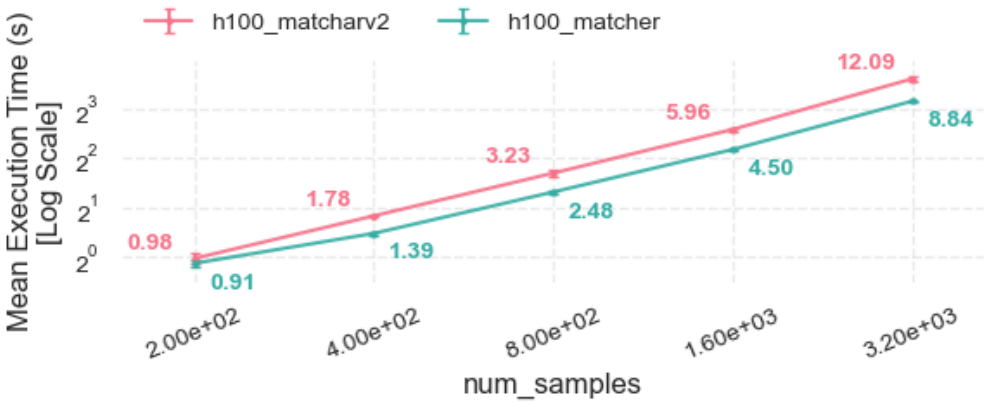

Figure 6: Varying number of samples, H100, `matcher` vs `matcharv2` (mean ± SD)

4 Bonus

After implementing `matcher`, we knew that the kernel `kerPreprocessMatchas` was occupying at least 99% of GPU execution time, and the majority of its instructions were spent matching each 3-word sample window across all signatures. We deduced that this could be avoided if the signatures were aligned in a way such that a few consecutive characters satisfied certain criteria. For example, we could perhaps find instances of the pattern 'AAAA' within each signature and record that position as the start point. This way, there would no longer be a need to scan through all signatures if the sample window did not start with 'AAAA'.

However, as this approach was probabilistic, there were multiple issues: (1) there was no guarantee all, or even any, signature would contain the pattern, especially if the probability was low; (2) if the probability was high, warp divergence would take back whatever benefits it might have offered; (3) dealing with 'N' required additional care, as a pattern containing 'N' in the signature might not be in the matching sample, or a pattern containing 'A/C/G/T' in the signature might not pick up 'N' in the sample.

To deal with (1), we decided on a compromise: if there was no matching pattern within a signature, then `kerPotentialMatchas` would simply iterate through it as usual. Otherwise, it would be added to the set of signatures with a matching pattern, which would all be skipped if the pattern was not found. With (2) put into consideration, we then sought to balance sparsity, to minimize warp divergence, with density, to maximize incidence within signatures. Furthermore, to address (3), we made the conclusion that two patterns were actually needed. One pattern had to exclude all instances of 'N', which would be applied to the signature, while the other had to include all 'N', which would be applied to the sample. Furthermore, we had to identify viable patterns, that worked optimally for varying 'N' ratios.

To achieve this, we ran a simulator on Python, which would estimate the likelihood of each signature containing the given pattern, the probability of warp divergence, and the overall gain:

$$\text{Compute Speedup} = \frac{\text{num sigs}}{1 + P(\text{warp div.}) \cdot \text{num sigs} + (1 - P(\text{warp div.})) \cdot P(\text{no pattern}) \cdot \text{num sigs}}$$

Based on our findings, we settled on the pattern 1 'C/T' followed by 4 'C'. As `kerPotentialMatchas` now used 41 active registers, which was previously 32, we changed the block size from `1024` to `512` and the grid size from `2 * num_SMs` to `3 * num_SMs` to maximize occupancy. Finally, we observe a speedup of up to 1.35 for bigger inputs. While this seemingly contradicts the 488.29% as suggested by the simulation, this stems from the fact that the process was still largely memory-bound. Additionally, we can estimate that the compute portion of the previous `kerPreprocessAllSigs` took up at least $\approx \frac{488.29(1-1.35)}{1.35(488.29-100)} = 32.6\%$ of its execution time.

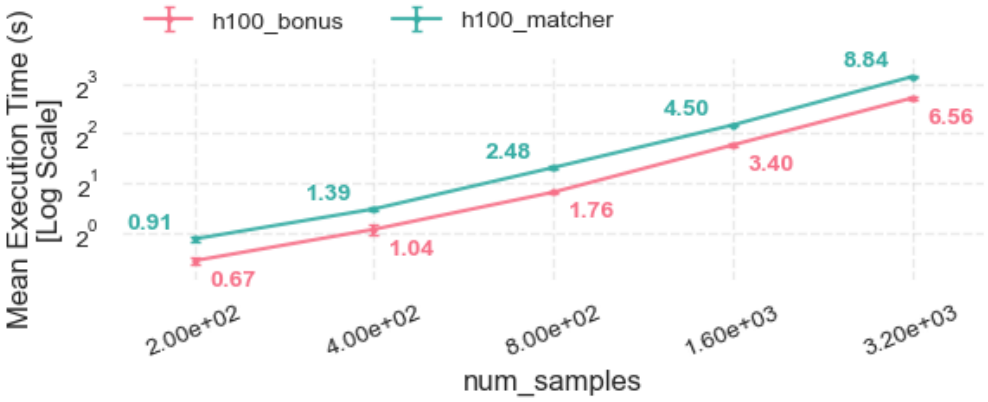

Figure 7: Varying number of samples, H100, `bonus` vs `matcher` (mean ± SD)

A Procedure

A.1 Experiment Generation

We generate all input files for a single experiment by running the following command:

```
./run_batch_gen.sh <experiment_name> <flag_to_sweep> "<values>" [overrides]
```

Variable
Number of signatures
Average signature length (×0.462 min, ×1.538 max)
Number of samples
Average sample length (×0.667 min, ×1.333 max)
Max viruses per sample (min viruses = 1)
Sample percentage with virus
'N' ratio, for both signature & sample

Flag	(Default Value)
n-sg	(210)
l-sg	(6500)
n-sp	(400)
l-sp	(1500000)
v	(10)
p	(0.01)
n	(0.1)

After running the shell script, the corresponding FASTA and FASTQ files are generated, and the names of each FASTQ-FASTA argument pair are then saved in a manifest file specified by `experiment_name`, under `experiments/`. For example, the following generates the input files in the user-specified range of number of samples, the user-specified number of signatures, and the default values for all other arguments:

```
./run_batch_gen.sh vary_num_samples n-sp "200 400 800 1600 3200" -n-sg 840
```

A.2 Obtaining Results

After generating the input files for an experiment, we can then specify the partition and the task with the following command:

```
./run_profile_suite.sh <program_name> <manifest_file> <a100|h100|all> <task-type>
```

task-type	
time	Runs each input pair 10 times on repeat. Records the times of each run to a single file reports/time_<fastq-name>.txt. Saves an output to outputs/<fastq-name>.txt for correctness checking.
ncu	Profiles each input pair on Nsight Compute. Saves the report to reports/ncu_<fastq-name>.ncu-rep.
nsys	Profiles each input pair on Nsight Systems. Saves the report to reports/nsys_<fastq-name>.nsys-rep.

For example, if we want to run the program `matchar` for the inputs generated in the previous example, on the H100 (xgpi) partition, and record the execution time repeatedly, we run:

```
./run_profile_suite.sh matchar experiments/vary_num_samples h100 time
```

B Data

Raw data is available on Google Sheets.

C AI Declaration

Google Gemini was used for the following components:

- Generating code for the main program. Specific links can be found within the relevant code files.
- Generating shell scripts: `https://gemini.google.com/share/d06a017f08d4`
- Generating graphing program in Python: `https://gemini.google.com/share/ea8c379e7eea`